

Chapitre 8 : Jointure entre les tables

I. Introduction

Ce chapitre présente le principe de la jointure entre les tables. Dans ce cours, on va s'intéresser seulement à l'équijointure.

II. Jointure

1. Définition

Les jointures permettent d'extraire des données issues de plusieurs tables. Le processus de normalisation du modèle relationnel est basé sur la décomposition et a pour conséquence d'augmenter le nombre de tables d'un schéma. Ainsi, la majorité des requêtes utilisent des jointures nécessaires pour pouvoir extraire des données de tables distinctes.

Une jointure est un produit cartésien de deux tables. Elle met en relation deux tables sur la base d'une clause de jointure (comparaison de colonnes). Généralement, cette comparaison fait intervenir une clé étrangère d'une table avec une clé primaire d'une autre table (car le modèle relationnel est fondamentalement basé sur les valeurs).

En SQL, l'expression d'une jointure se fait en précisant le nom des colonnes des tables sur lesquelles on fait la jointure, en désignant les colonnes des différentes tables en écrivant le nom de la table, suivie d'un point puis du nom de la colonne. La clause *WHERE* permet de préciser la qualification de la jointure.

2. Types de Jointure

Bien que, dans le vocabulaire courant, on ne parle que de « jointures » en fonction de la nature de l'opérateur utilisé dans la requête, de la clause de jointure et des tables concernées, on distingue :

- Les jointures internes (inner joins) :
 - L'équijointure (equi join) est la plus connue, elle utilise l'opérateur d'égalité dans la clause de jointure. La jointure naturelle est conditionnée en plus par le nom des colonnes. La non équijointure utilise l'opérateur d'inégalité dans la clause de jointure. –
 - L'autojointure (self join) est un cas particulier de l'équijointure, qui met en œuvre deux fois la même table (des alias de tables permettront de distinguer les enregistrements entre eux).

▪ La jointure externe (outer join), la plus compliquée, qui favorise une table (dite « dominante ») par rapport à l'autre (dite « subordonnée »). Les lignes de la table dominante sont retournées même si elles ne satisfont pas aux conditions de jointure. Le tableau suivant illustre cette classification sous la forme de quelques conditions appliquées :

Type de jointure	Syntaxe de la condition
Équijointure	...WHERE comp = compa;
Autojointure	...WHERE alias1.chefPil = alias2.brevet;
Jointure externe	...FROM Compagnie LEFT OUTER JOIN Pilote ON comp = compa;

Dans ce cours, on va s'intéresser seulement aux deux premiers types de jointures.

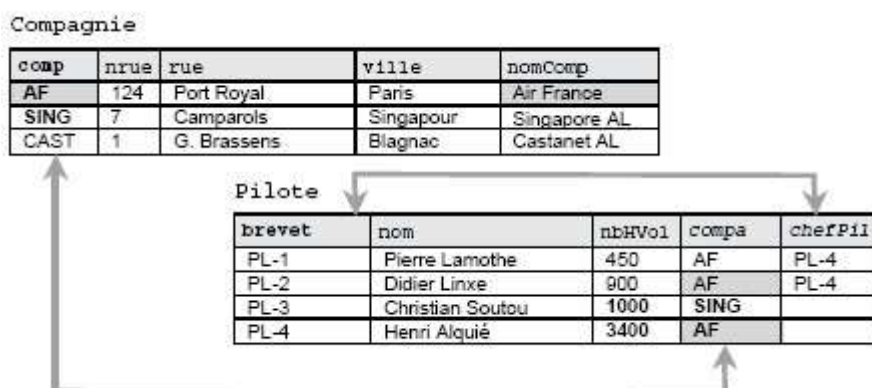
1. Équijointure

Une équijointure utilise l'opérateur d'égalité dans la clause de jointure et compare généralement des clés primaires avec des clés étrangères.

En considérant les tables suivantes, les équijointures se programment soit sur les colonnes comp et compa soit sur les colonnes brevet et chefPil. Extrayons par exemple :

- l'identité des pilotes de la compagnie de nom 'Air France' ayant plus de 500 heures de vol (requête R1) ;
- les coordonnées des compagnies qui embauchent des pilotes de moins de 500 heures de vol (requête R2).

La jointure qui résoudra la première requête est illustrée dans la figure par les données grisées, tandis que la deuxième jointure est représentée par les données en gras.



Écriture « relationnelle »

- MySQL recommande d'utiliser des alias de tables pour améliorer les performances.
- Les alias sont obligatoires quand des colonnes de différentes tables portent le même nom ou dans le cas d'autojointures.

Écriture « SQL2 »

- La clause JOIN...ON condition permet de programmer une équijointure.

- L'utilisation de la directive INNER devant JOIN... est optionnelle et est appliquée par défaut.

Le tableau suivant détaille ces requêtes avec les deux syntaxes. Les clauses de jointures sont grisées.

Requête	Jointure relationnelle	Jointure SQL2
R1	<pre>SELECT brevet, nom FROM Pilote, Compagnie WHERE comp = compa AND nomComp = 'Air France' AND nbHVol > 500;</pre>	<pre>SELECT brevet, nom FROM Compagnie JOIN Pilote ON comp = compa WHERE nomComp = 'Air France' AND nbHVol > 500;</pre>
	<pre>+-----+-----+ brevet nom +-----+-----+ PL-2 Didier Linxe PL-4 Henri Alquié +-----+-----+</pre>	
R2	<pre>SELECT cpg.nomComp, cpg.nrue, cpg.rue, cpg.ville FROM Pilote pil, Compagnie cpg WHERE cpg.comp = pil.compa AND pil.nbHVol < 500;</pre>	<pre>SELECT nomComp, nrue, rue, ville FROM Compagnie INNER JOIN Pilote ON comp = compa WHERE nbHVol < 500;</pre>
	<pre>+-----+-----+-----+-----+ nomComp nrue rue ville +-----+-----+-----+-----+ Air France 124 Port Royal Paris +-----+-----+-----+-----+</pre>	

2. Autojointure

Cas particulier de l'équijointure, l'autojointure relie une table à elle-même.

Extrayons par exemple :

- L'identité des pilotes placés sous la responsabilité des pilotes de nom 'Alquié' (requête R3) ;
- La somme des heures de vol des pilotes placés sous la responsabilité des chefs pilotes de la compagnie de nom 'Air France' (requête R4).

Ces requêtes doivent être programmées à l'aide d'une autojointure, car elles imposent de parcourir deux fois la table Pilote (examen de chaque pilote en le comparant à un autre).

Les autojointures sont réalisées entre les colonnes brevet et chefPil.

La jointure de la première requête est illustrée dans la figure par les données surlignées en clair, tandis que la deuxième jointure est mise en valeur par les données surlignées en foncé.

Compagnie

comp	nrue	rue	ville	nomComp
AF	124	Port Royal	Paris	Air France
SING	7	Camparols	Singapour	Singapore AL
CAST	1	G. Brassens	Blagnac	Castanet AL

Pilote

brevet	nom	nbHVol	compa	chefPil
PL-1	Pierre Lamothe	450	AF	PL-4
PL-2	Didier Linxe	800	AF	PL-4
PL-3	Christian Soutou	1000	SING	
PL-4	Henri Alquié	3400	AF	

Le tableau suivant détaille ces requêtes, les clauses d'autojointures sont surlignées. Dans les deux syntaxes, il est impératif d'utiliser un alias de table. Concernant l'écriture « SQL2 », on remarque que les clauses JOIN s'imbriquent (pour joindre plus de deux tables).

Requête	Jointure relationnelle	Jointure SQL2
R3	<pre>SELECT p1.brevet, p1.nom FROM Pilote p1, Pilote p2 WHERE p1.chefPil = p2.brevet AND p2.nom LIKE '%Alquié%';</pre>	<pre>SELECT p1.brevet, p1.nom FROM Pilote p1 JOIN Pilote p2 ON p1.chefPil = p2.brevet WHERE p2.nom LIKE '%Alquié%';</pre>
	<pre>+-----+-----+ brevet nom +-----+-----+ PL-1 Pierre Lamothe PL-2 Didier Linxe +-----+-----+</pre>	
R4	<pre>SELECT SUM(p1.nbHVol) FROM Pilote p1, Pilote p2, Compagnie cpg WHERE p1.chefPil = p2.brevet AND cpg.comp = p2.compa AND cpg.nomComp = 'Air France';</pre>	<pre>SELECT SUM(p1.nbHVol) FROM Pilote p1 JOIN Pilote p2 ON p1.chefPil = p2.brevet JOIN Compagnie ON comp = p2.compa WHERE nomComp = 'Air France';</pre>
	<pre>+-----+ SUM(p1.nbHVol) +-----+ 1350.00 +-----+</pre>	

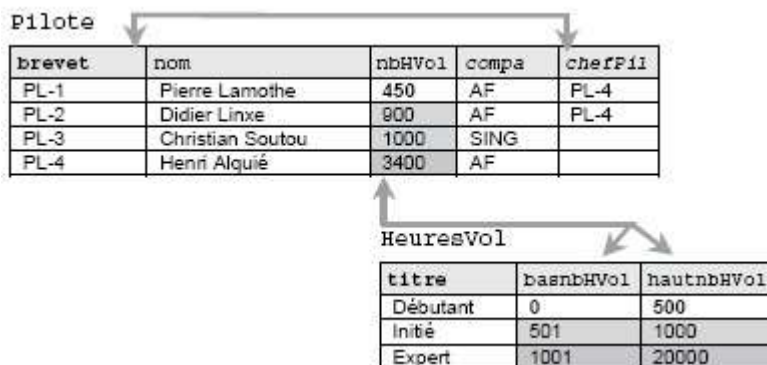
Inéquijointure

Les requêtes d'inéquijointures font intervenir tout type d'opérateur (<>, >, <, >=, <=, BETWEEN, LIKE, IN). À l'inverse des éqijointures, la clause d'une inéquijointure n'est pas basée sur l'égalité de clés primaires (ou candidates) et de clés étrangères.

En considérant les tables suivantes, extrayons par exemple :

- Les pilotes ayant plus d'expérience que le pilote de numéro de brevet 'PL-2' (requête R5).
- Le titre de qualification des pilotes en raisonnant sur la comparaison des heures de vol avec un ensemble de référence, ici la table HeuresVol (requête R6). Dans notre exemple, il s'agit par exemple de retrouver le fait que le premier pilote est débutant.

La jointure qui résoudra la deuxième requête est illustrée par les niveaux de gris.



Le tableau suivant détaille ces requêtes, les clauses d'inéquijointures sont surlignées :

Requête	Jointure relationnelle	Jointure SQL2												
R5	<pre>SELECT p1.brevet, p1.nom, p1.nbHVol, p2.nbHVol "Référence" FROM Pilote p1, Pilote p2 WHERE p1.nbHVol > p2.nbHVol AND p2.brevet = 'PL-2';</pre>	<pre>SELECT p1.brevet, p1.nom, p1.nbHVol, p2.nbHVol "Référence" FROM Pilote p1 JOIN Pilote p2 ON p1.nbHVol>p2.nbHVol WHERE p2.brevet = 'PL-2';</pre>												
	<table><tr><th>brevet</th><th>nom</th><th>nbHVol</th><th>Référence</th></tr><tr><td>PL-3</td><td>Christian Soutou</td><td>1000.00</td><td>900.00</td></tr><tr><td>PL-4</td><td>Henri Alquié</td><td>3400.00</td><td>900.00</td></tr></table>	brevet	nom	nbHVol	Référence	PL-3	Christian Soutou	1000.00	900.00	PL-4	Henri Alquié	3400.00	900.00	
brevet	nom	nbHVol	Référence											
PL-3	Christian Soutou	1000.00	900.00											
PL-4	Henri Alquié	3400.00	900.00											

Requête	Jointure relationnelle	Jointure SQL2																				
R6	<pre>SELECT pil.brevet, pil.nom, pil.nbHVol, hv.titre FROM Pilote pil, HeuresVol hv WHERE pil.nbHVol BETWEEN hv.basnbHVol AND hv.hautnbHVol;</pre>	<pre>SELECT brevet, nom, nbHVol, titre FROM Pilote JOIN HeuresVol ON nbHVol BETWEEN basnbHVol AND hautnbHVol;</pre>																				
	<table><tr><th>brevet</th><th>nom</th><th>nbHVol</th><th>titre</th></tr><tr><td>PL-1</td><td>Pierre Lamothe</td><td>450.00</td><td>Débutant</td></tr><tr><td>PL-2</td><td>Didier Linxe</td><td>900.00</td><td>Initié</td></tr><tr><td>PL-3</td><td>Christian Soutou</td><td>1000.00</td><td>Initié</td></tr><tr><td>PL-4</td><td>Henri Alquié</td><td>3400.00</td><td>Expert</td></tr></table>		brevet	nom	nbHVol	titre	PL-1	Pierre Lamothe	450.00	Débutant	PL-2	Didier Linxe	900.00	Initié	PL-3	Christian Soutou	1000.00	Initié	PL-4	Henri Alquié	3400.00	Expert
brevet	nom	nbHVol	titre																			
PL-1	Pierre Lamothe	450.00	Débutant																			
PL-2	Didier Linxe	900.00	Initié																			
PL-3	Christian Soutou	1000.00	Initié																			
PL-4	Henri Alquié	3400.00	Expert																			

III. Les Opérations Ensemblistes

Les opérations ensemblistes en SQL, sont réalisées grâce aux opérateurs:

- UNION
- INTERSECT
- EXCEPT

Ces opérateurs s'utilisent entre deux clauses *SELECT*.

1. L'opérateur UNION

Cet opérateur permet d'effectuer une UNION des tuples sélectionnés par deux clauses *SELECT*.

```
SELECT ---- FROM ---- WHERE -----
```

```
UNION
```

```
SELECT ---- FROM ---- WHERE -----
```

Par défaut les doublons sont automatiquement éliminés. Pour conserver les doublons, il est possible d'utiliser une clause *UNION ALL*.

2. L'opérateur INTERSECT

Cet opérateur permet d'effectuer une INTERSECTION des tuples sélectionnés par deux clauses *SELECT*.

```
SELECT ---- FROM ---- WHERE -----
```

```
INTERSECT
```

```
SELECT ---- FROM ---- WHERE -----
```

L'opérateur *INTERSECT* n'étant pas implémenté dans tous les SGBD, il est possible de le remplacer par des commandes usuelles :

```
SELECT a,b FROM table1
```

```
WHERE EXISTS (SELECT c, d FROM table2
```

```
WHERE a=c AND b=d );
```

3. L'opérateur EXCEPT

Cet opérateur permet d'effectuer une DIFFERENCE entre les tuples sélectionnés par deux clauses *SELECT*, c'est-à-dire sélectionner les tuples de la première table n'appartenant pas à la seconde.

```
SELECT a,b FROM table1 WHERE -----
```

```
EXCEPT
```

```
SELECT c,d FROM table2 WHERE -----
```

L'opérateur *EXCEPT* n'étant pas implémenté dans tous les SGBD, il est possible de le remplacer par des commandes usuelles:

```
SELECT a,b FROM table1
```

```
WHERE NOT EXISTS (SELECT c,d FROM table2
```

```
WHERE a=c AND b=d );
```